

MINISTRY OF EDUCATION AND TRAINING
HUNG YEN UNIVERSITY OF TECHNOLOGY AND EDUCATION



FINAL PROJECT

WEB APPLICATION

**WEB APPLICATION DEVELOPMENT FOR E-
COMMERCE PLATFORM MANAGEMENT**

MAJOR: COMPUTER SCIENCE

STUDENT: VU QUANG PHUC

STUDENT ID: 12522079

CLASS: 124221KS

INSTRUCTOR: Ph.D VU XUAN THANG

HUNG YEN – 2025

COMMENTS

Comments from instructor:

This image shows a full page of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

INSTRUCTOR

(Signature and Full Name)

Vu Xuan Thang

COMMITMENT

I hereby certify that the project titled "Web Application Development for E-Commerce Platform Management " is the result of my own work conducted under the guidance of my course instructor Vu Xuan Thang.

All referenced materials used in this project have been clearly cited in the bibliography section. The results presented in this project and the developed program are entirely my own work.

If I violate this declaration, am I prepared to accept full responsibility before the faculty and university

Hung Yen, June, 12 , 2025

Student

Vu Quang Phuc

ACKNOWLEDGEMENT

First and foremost, I would like to express my sincere gratitude to the Department of Software Engineering, Faculty of Information Technology – Hung Yen University of Technology and Education, for providing favorable conditions that enabled me to complete this major project.

I would especially like to extend my heartfelt thanks to my course instructor Vu Xuan Thang for his dedicated guidance and support throughout the entire process of this project.

I am also deeply grateful to all the lecturers and faculty members at the university who have imparted invaluable knowledge and skills that have been essential for the successful completion of this project.

Although I have made every effort, given my limited experience, there may still be shortcomings in the execution of this project. I sincerely hope to receive constructive feedback and suggestions from the professors and instructors to improve the results presented in this work.

Thank you very much!

TABLE OF CONTENTS

COMMITMENT	1
ACKNOWLEDGEMENT	2
GLOSSARY OF TERMS	5
LIST OF TABLES	6
LIST OF FIGURES.....	7
CHAPTER 1: GENERAL INTRODUCTION	8
1.1 Motivation	8
1.2 Objectives	8
1.3 Limitations and Scope	9
1.4 Implementations	10
1.5 Methodology	11
CHAPTER 2: THEORICAL BACKGROUND	12
2.1 Software Development Life Cycle	12
2.2 Object-Oriented Analysis and Design.....	13
2.3 Front-end Design and Programming	15
2.4 Back-end Design and Programming	17
2.5 Data Manipulation Technologies	19
2.6 Three-Layer Model.....	19
CHAPTER 3: SYSTEM ANALYSIS AND DESIGN	22
3.1 Software requirements specification	22
3.2 System design.....	28
CHAPTER 4: WEBSITE IMPLEMENTATION	34
4.1 User features implementation.....	34

4.2	Admin features implementation	40
4.3	Testing and Deploying the Application	44
	CONCLUSION	45
	REFERENCES.....	46

GLOSSARY OF TERMS

Index	Term	Full form	Meaning
1	MVC	Model View Control	
2	GUI	Graphical User Interface	

LIST OF TABLES

Table 3.1 Administrator Functionalities	22
Table 3.2 Entity classes.....	25
Table 3.3 Non-functional requirements	27
Table 3.4 Account table	28
Table 3.5 Product table	29
Table 3.6 Shop table.....	30
Table 3.7 Comment table	30
Table 3.8 Voucher table	31
Table 3.9 Oder table	32
Table 3.10 Item table.....	32
Table 3.11 Category table	33

LIST OF FIGURES

Figure 4.1 Homepage User Page.....	34
Figure 4.2 Login User Page	35
Figure 4.3 Register User Page.....	35
Figure 4.4 Cart User Page	36
Figure 4.5 Account User Page	36
Figure 4.6 Search User Page	37
Figure 4.7 Checkout User Page.....	37
Figure 4.8 Bill User Page.....	38
Figure 4.9 Feedback User Page.....	38
Figure 4.10 About Page.....	39
Figure 4.11 Product management GUI	40
Figure 4.12 Coupon management GUI	41
Figure 4.13 Feedback management GUI	41
Figure 4.14 Delivery management GUI.....	42
Figure 4.15 Account management GUI	42
Figure 4.16 Category management GUI	43
Figure 4.17 Statistic GUI	43

CHAPTER 1: GENERAL INTRODUCTION

1.1 Motivation

I have a strong passion for e-commerce platform management and am eager to deepen my understanding of the technologies that power successful online businesses. This project, as part of my web development course, provides an ideal opportunity to apply theoretical knowledge in a practical setting by building a functional e-commerce platform.

Through this project, I aim to enhance my skills in web technologies such as front-end development, back-end programming, database design, and user interface optimization. I am particularly interested in how e-commerce platforms manage complex processes like product listing, inventory control, customer interactions, and secure transactions.

By developing this platform, I hope to gain hands-on experience in integrating various technologies and frameworks, improving my problem-solving abilities, and understanding the challenges involved in creating scalable, user-friendly online systems. This experience will be invaluable as I pursue a career in e-commerce and web application development.

1.2 Objectives

1.2.1 *General objectives*

The general objective of this project is to develop a complete and efficient management system for an e-commerce platform to effectively address the challenges in managing and operating the marketplace.

1.2.2 *Specific objectives*

- Develop an intelligent and flexible system capable of adapting dynamically to customer needs and business changes.
- Optimize resource utilization such as product inventory, supplier management, and customer service to minimize waste and enhance operational efficiency.

- Enhance service quality and customer experience by providing detailed, reliable, and easily accessible product information, while streamlining the processes of product search, order placement, payment, and delivery.
- Strengthen user management by implementing effective solutions for managing customers, sellers, and administrators.
- Build a modern, user-friendly software system that facilitates efficient and flexible management and operation of the e-commerce marketplace.

1.3 Limitations and Scope

1.3.1 Research subject

- Research Subject: The management and operational system of the e-commerce platform. (This refers to the overall system and processes involved in managing and operating the e-commerce marketplace.)
- Research Object: The customers who use the platform, service providers (such as sellers and logistics partners), and the platform's management personnel. (These are the specific entities or groups that the research focuses on within the system, including buyers, sellers, service providers, and administrators.)

1.3.2 Research Scope

The research will be conducted over a period of three months, including one month for collecting secondary data and two months for carrying out the study. The project "E-commerce Platform Management" not only aims to contribute scientific research value but also to provide practical solutions to improve business operations within the e-commerce sector.

Specifically, the study will focus on:

- Analyzing the operational and management processes of the e-commerce platform.
- Identifying challenges and inefficiencies in current marketplace management.

- Proposing and evaluating solutions to optimize resource utilization such as product inventory, supplier coordination, and customer service.
- Enhancing user experience by streamlining order processing, payment, and delivery workflows.
- Strengthening management of platform stakeholders including buyers, sellers, and administrators.
- Developing a modern, scalable, and user-friendly system to support effective and flexible platform management.

This scope ensures that the research delivers both theoretical insights and actionable recommendations to boost the efficiency and competitiveness of e-commerce marketplaces.

1.4 Implementations

By applying the software development life cycle and leveraging the comprehensive knowledge and skills acquired from previous courses, the project “Building an E-commerce Management Website” will carry out the following tasks:

- ***System Survey and Analysis:***

Conduct detailed research to understand the requirements, business processes, and challenges of managing an e-commerce platform.

- ***Design and Specification of System Functions:***

Define and document the functional requirements and features of the system, including user roles, product management, order processing, payment, and reporting.

- ***Database Construction:***

Design and implement the database schema to store all necessary data such as products, users, orders, and transactions securely and efficiently.

- ***Interface Design and Specification:***

Create wireframes and prototypes for the user interface to ensure a user-friendly and intuitive experience for administrators, sellers, and buyers.

- ***System Programming:***

Develop the front-end and back-end components of the website using appropriate technologies and frameworks, ensuring functionality, responsiveness, and security.

➤ ***System Deployment and Testing:***

Deploy the system in a real or simulated environment for testing, including functional testing, usability testing, and performance evaluation to ensure the system meets all requirements and operates smoothly.

1.5 Methodology

During the implementation of the project “Building an E-commerce Platform Management Website”, the following research methods were applied:

➤ ***Practical Observation Method:***

Observing the real operations of staff working on the e-commerce platform, including direct interaction with the platform as a user (e.g., browsing, purchasing). Conducting surveys and interviews with platform managers, sellers, and customer service representatives to understand workflows and processes.

➤ ***Information Collection Method:***

Gathering information by analyzing related e-commerce websites and mobile applications to benchmark features, user experience, and management practices.

➤ ***Analysis and Comparison Method:***

Analyzing the collected data and observations, then comparing them with existing systems and industry standards to develop ideas for designing and building the software system.

CHAPTER 2: THEORETICAL BACKGROUND

2.1 Software Development Life Cycle

The software development process is a series of structured steps carried out to create software. It ensures that the software is developed efficiently and meets user requirements.

Why apply a software development process?

- It clearly defines project goals, scope, resources, timeline, and costs, reducing risks and increasing the chance of success.
- It improves communication and collaboration among stakeholders such as customers, developers, users, and managers.
- It enforces professional standards, techniques, and tools to produce high-quality, secure, and reliable software products.
- It enables monitoring and controlling project progress, quality, and costs, allowing performance evaluation and corrective actions when needed.
- It supports software maintenance and upgrades over time to meet evolving customer and market needs.

The typical software development process consists of six main phases:

1. Planning and Requirements Analysis (Analysis):

Collect and analyze user and stakeholder requirements, study the market, and produce a detailed Software Requirements Specification (SRS) document outlining functional and non-functional requirements.

2. Software Design (Design):

Define the overall system architecture, design modules, interfaces, data structures, and select technologies, considering budget, time, and risks.

3. Development (Coding):

Implement the design by writing code for front-end and back-end components using appropriate programming languages and frameworks. Perform initial testing and debugging.

4. *Testing:*

Conduct thorough tests including unit, integration, system, and user acceptance testing to ensure the software meets specifications and is free of defects.

5. *Deployment:*

Release the software into the production environment for end-users, ensuring proper installation and configuration.

6. *Maintenance:*

Provide ongoing support by fixing bugs, updating features, and improving performance to adapt to changing requirements and environments.

2.2 Object-Oriented Analysis and Design

Unified Modeling Language (UML) is a standardized visual modeling language designed to represent object-oriented systems. Its main purposes include:

- Modeling systems using object-oriented concepts.
- Bridging human understanding with the events that need to be modeled.
- Addressing issues of inheritance levels in complex systems with various constraints.
- Creating a modeling language usable by both humans and machines.

UML provides a set of graphical notations to specify, visualize, construct, and document software systems. It helps improve communication among developers and stakeholders, standardizes system design, and supports the entire software development life cycle. UML diagrams represent both the structural and behavioral

aspects of systems, making it easier to understand and manage complex software projects.

2.2.1 Use Case Diagram

A Use Case Diagram is a type of diagram within the Unified Modeling Language (UML) that visually represents the functional requirements of a system. It illustrates what the system must do to meet the needs of its users. Rather than describing how the system operates internally, it focuses on the interactions between users (called actors) and the system from a high-level perspective.

2.2.2 Class Diagram

A class diagram, used in software engineering and object-oriented design, visually represents the structure of a system by showing its classes, attributes, and relationships. It's a blueprint of the system, highlighting the classes, their properties (attributes), and the operations they provide (methods). Essentially, it's a visual representation of how different parts of a system are organized and connected.

2.2.3 Sequence Diagram

A sequence diagram, or interaction diagram in Unified Modeling Language (UML), visually represents the interactions between objects in a system over time. It depicts objects as lifelines, with messages exchanged between them shown as arrows, illustrating the order and timing of these interactions.

2.2.4 State Machine Diagram

A State Machine Diagram is a visual representation that shows how an object or system transitions between different states in response to events. It's a way to model the behavior of a system by defining its possible states, the events that cause transitions between those states, and the actions that result from those transitions.

2.2.5 Activity Diagram

Activity diagram is another important behavioral diagram in UML diagram to describe dynamic aspects of the system. Activity diagram is essentially an advanced version of flow chart that modeling the flow from one activity to another activity.

2.2.6 Component Diagram

A component diagram in Unified Modeling Language (UML) is a diagram that visualizes the structure of a system by showing how its components are organized and how they relate to each other. It helps to understand the high-level organization and dependencies between different parts of a system.

2.2.7 Deployment Diagram

Deployment diagrams are a type of UML diagram that showcases the physical deployment of artifacts on nodes. To put it simply, they describe the hardware components where software components are installed.

2.3 Front-end Design and Programming

The front-end—also known as the client-side—refers to the portion of a web application or website that is directly visible and interactive for end-users. It encompasses all components related to user interface design, layout, and user interaction. Front-end development involves both the visual presentation of a site and the implementation of interactive functionality using client-side technologies.

Core Technologies in Front-End Development

To become a proficient front-end developer, one must gain a solid understanding of several fundamental technologies and tools, as outlined below:

- ***HTML (Hypertext Markup Language):***

HTML is the foundational markup language used to define the structure and content of web pages. It determines how various elements such as text, images, and links are organized and displayed on the browser.

- ***CSS (Cascading Style Sheets):***

CSS is used to style and format HTML elements. It controls the visual appearance, including colors, fonts, spacing, layout, and responsive design aspects. CSS plays a vital role in enhancing the user interface and improving usability.

- ***JavaScript:***

JavaScript is the primary programming language for the client side. It enables developers to implement dynamic behaviors and interactive features on web pages, such as form validation, animations, and real-time content updates without requiring a page reload.

Advanced Front-End Libraries and Frameworks

In addition to the core technologies above, modern front-end development often involves the use of libraries and frameworks that simplify and accelerate the development process. These include:

- ***React (ReactJS):***

React is an open-source JavaScript library developed by Facebook, designed specifically for building user interfaces in a component-based architecture. React focuses solely on the view layer of the application and facilitates the creation of reusable UI components, enabling more maintainable and scalable front-end architectures.

- ***Bootstrap:***

Bootstrap is a widely-used front-end framework that combines predefined CSS classes and JavaScript components. It streamlines the process of building responsive and mobile-first websites. Bootstrap includes templates for forms, buttons, navigation bars, and other UI elements, enabling rapid prototyping and consistency in design.

- ***Tailwind CSS:***

Tailwind CSS is a utility-first CSS framework that provides low-level utility classes to construct custom designs directly in HTML. Unlike traditional frameworks, Tailwind allows developers to build highly customizable and scalable UI components without writing custom CSS from scratch. It offers flexibility and promotes a more atomic approach to styling.

- ***jQuery:***

jQuery is a lightweight JavaScript library that simplifies DOM manipulation, event handling, and AJAX operations across different browsers. While its usage has declined with the advent of modern JavaScript frameworks, it remains useful for quick development tasks and legacy systems.

User Experience (UX) and User Interface (UI) Considerations

Front-end development is not limited to technical implementation but also involves understanding UX/UI design principles. A successful front-end developer should be familiar with:

- ***UI Design:*** The process of designing visually appealing and consistent interfaces that align with branding and user expectations.
- ***UX Design:*** Ensuring optimal user experiences by focusing on usability, accessibility, responsiveness, and overall satisfaction when interacting with the web application.

Effective UI/UX design enhances user engagement, reduces bounce rates, and contributes to the overall success of a web-based system.

2.4 Back-end Design and Programming

Node.js, often referred to simply as NodeJS, has become a leading technology in the development of modern web applications, particularly due to its scalability and performance. As of recent global estimates, approximately 2% of all web applications—roughly 22 million projects—leverage Node.js for backend services.

Node.js is an open-source JavaScript runtime environment built on Google Chrome's V8 engine. It enables developers to execute JavaScript code on the server side. One of the most prominent advantages of Node.js is its non-blocking, event-driven architecture, which allows for real-time processing and asynchronous communication—key requirements for high-traffic e-commerce platforms.

In essence, Node.js allows developers to write the backend logic using the same language used on the frontend (JavaScript), thereby simplifying development workflows and encouraging a full-stack JavaScript environment.

Node.js Backend in E-Commerce Applications

In the context of an e-commerce platform, the Node.js backend handles all server-side operations, including data processing, business logic, and database interaction. It serves as the core engine behind the platform's functionality, supporting seamless shopping experiences, secure transactions, and efficient data handling.

Key responsibilities of a Node.js backend in an e-commerce system include:

1. Handling Client Requests

When a user browses products, adds items to a cart, or places an order, these actions are sent as HTTP requests from the client to the server. The Node.js backend receives these requests, processes them, and returns the appropriate response—such as displaying a product page or confirming an order.

2. Interacting with Databases

The backend frequently communicates with databases (e.g., MongoDB, PostgreSQL, MySQL) to store and retrieve data. Examples include fetching product information, storing user credentials, retrieving order history, or updating inventory after a purchase. Node.js offers various libraries and ORMs (Object Relational Mappers) like Mongoose and Sequelize to facilitate this interaction efficiently.

3. Performing Business Logic and Calculations

Complex server-side operations—such as calculating total order values, applying discount codes, determining shipping fees, or validating payment status—are executed by the backend. Node.js handles these computations quickly and efficiently, even under high concurrency.

2.5 Data Manipulation Technologies

In the development of the backend for the e-commerce platform, Node.js is employed as the core server-side runtime environment, while MySQL serves as the relational database management system (RDBMS). This combination offers a powerful and scalable solution for managing data-driven operations in an online shopping system.

To enable seamless interaction between the Node.js server and the MySQL database, the project utilizes the `mysql2` library—one of the most widely adopted Node.js packages for efficient database connectivity and data manipulation. This library supports both basic SQL queries and advanced features such as prepared statements, connection pooling, and promise-based APIs, making it a flexible tool for modern web applications.

2.6 Three-Layer Model

The Three-Layer Architecture, also known as the 3-Layer Model, is a widely adopted software design pattern used to separate concerns within an application by organizing its functionality into distinct layers. Each layer is responsible for a specific aspect of the system, promoting modularity, maintainability, and scalability—particularly important when developing large-scale systems such as e-commerce platforms.

This architecture enhances the manageability and robustness of applications by preventing overlapping responsibilities, avoiding data inconsistency, and improving error handling. It is especially effective in complex systems where clean separation of duties is essential for both team collaboration and future extensibility.

1. *Presentation Layer (UI/GUI Layer)*

This layer is responsible for interacting directly with the user. It includes all visual elements and interface components (e.g., web pages, forms, buttons) that allow users to perform operations such as browsing products, adding items to the cart, or checking out.

Key functions:

- Data input and output: Collects input from users and displays responses.
- Validation: Performs client-side validation of data before passing it to the business logic layer.
- User experience (UX): Ensures intuitive navigation and responsiveness across devices.

In an e-commerce context, this would typically be built using technologies such as HTML, CSS, JavaScript, ReactJS, or Vue.js.

2. *Business Logic Layer (BLL)*

The Business Logic Layer acts as the intermediary between the user interface and the data layer. It contains the core rules and workflows that govern how the system operates.

Its responsibilities include:

- Receiving input data from the presentation layer and applying business rules (e.g., applying discounts, validating inventory availability).
- Performing operations such as price calculations, transaction validations, and promotion logic.
- Enforcing constraints and ensuring data integrity before interacting with the data access layer.
- Returning processed results to the UI for display.

This layer can be implemented in Node.js, where the application logic is defined in controllers and service modules.

3. Data Access Layer (DAL)

The Data Access Layer handles all operations involving the database. It is responsible for communicating with the database system (in this case, MySQL) and executing operations such as insertions, deletions, updates, and queries.

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

3.1 Software requirements specification

3.1.1 Functional Requirement

a) Administrator

Table 3.1 Administrator Functionalities

No.	Feature Name	Description
I. Employee Management		
1	Add Employee Information	This feature allows the admin to add new employees to the system.
2	Edit Employee Information	This feature allows updating employee details. Only employees can use this feature.
3	Delete Employee Information	This feature allows the admin to delete employee data when they no longer work for the company.
4	Search Employee	This feature allows the admin to quickly find employees by name or birth date.
II. Customer Management		
1	Chat with Shop	This feature allows customers to chat directly with the shop about products and prices.
2	Search & Filter Products	This feature helps customers filter products based on criteria like price, views, categories, or ratings.
3	Update Personal Information	This feature allows users to edit their personal information.
4	Register Store	This feature helps customers register their own store on the platform.

No.	Feature Name	Description
5	Logout	This feature allows users to log out of the system.
6	Login	This feature allows users to log in to use role-based and business-specific features.
7	Register Account	This feature allows new users to create an account to start shopping.
III. Store Management		
1	Statistics Dashboard	Allows store owners to view store performance, product quantity, order count, and revenue.
2	Chat with Customers	The shop can chat with users similarly to how users chat with the shop.
IV. Product Approval Management		
1	Lock Product	Allows admin to lock a product if it's reported or violates policy.
2	Approve Product	Allows admin to approve a product for listing after verifying it's valid and meets all conditions.
V. Statistics & Export to Excel		
1	Customer Statistics	Allows admin to view user traffic data in the system.
2	Product Statistics	Displays statistics for products.
3	Order Statistics	Displays statistics for orders.
4	Revenue Statistics	Displays revenue statistics.
5	Store Statistics	Displays store registration statistics.
VI. Cart Management		

1	Add to Cart	Allows users to add products to their personal cart for future reference.
2	Remove from Cart	Allows users to remove items from their cart if no longer interested.
VII. Product Management		
1	Add Product	Allows sellers to add new products to the platform.
2	Delete Product	Allows sellers to delete products they no longer want to sell.
3	Edit Product	Allows sellers to edit product information such as price, quantity, and description.
VIII. Discount Code Management		
1	Add Discount Code	Allows store owners to create discount codes for marketing purposes.
2	Delete Discount Code	Allows store owners to delete expired or invalid discount codes.
3	Search Discount Code	Allows store owners to search for existing discount codes.
IX. Order Management		
1	Place Order	Allows customers to submit an order to the store.
2	Cancel Order	Allows customers to cancel an order if it has not yet been accepted by the seller.
3	Accept Order	Allows sellers to accept orders and proceed with packaging and delivery.
4	Reject Order	Allows sellers to reject orders if the item is unavailable or if necessary.

5	Receive Order	Allows buyers to confirm receipt of the delivered item.
X. Product Review Management		
1	Rate Product	Allows users to rate products from 1 to 5 stars and write comments after purchase.
2	Delete Product Review	Allows users to delete their submitted product reviews.
XI. Product Category Management		
1	Add Product Category	Allows admin to add new product categories.
2	Edit Product Category	Allows admin to update existing category names.
3	Delete Product Category	Allows admin to remove unwanted categories.

3.1.2 Entity class diagram

Table 3.2 Entity classes

No.	Class Name	Description
1	Account	This class is responsible for managing system login, defining basic information needed to restore account login like username, password, and methods of authentication and account verification.
2	Censor	This class defines the information of a reviewer who checks the actual status and uses functions such as updating the review status of products.

No.	Class Name	Description
3	Customer	This class defines user information in the system like name, age, date of birth, and provides methods such as placing orders, commenting, and shopping.
4	Shop	This class defines shop information, representing a shop entity outside the physical world and provides functions such as editing shop data, viewing shop information.
5	Shopping cart	This class defines the basic structure of a shopping cart with customer information and items, with methods to add, remove products.
6	Order	This class defines order information including methods for placing orders, canceling orders, updating order status.
7	Order detail	This class defines detailed order information, including product-specific status (confirmed, canceled, etc.).
8	Product	This class defines basic product information such as product name, quantity in stock, and methods like add, edit, delete, and update product status.
9	Category	This class defines information related to product categories, including category levels and management methods.
10	Comment	This class defines basic comment data including rating, user, content, and methods to add or remove comments.

No.	Class Name	Description
11	Message	This class allows users to chat, defines a message entity including sender, receiver, content, and supports methods for sending and deleting (not editing) messages.
12	Voucher	This class defines basic voucher information like name, discount, promotion details, and includes methods to add and delete (no edit).
13	Notification	This class defines a notification entity that is automatically triggered when a condition is met, including methods to add or delete notifications.

3.1.3 Non-functional requirements

Table 3.3 Non-functional requirements

No.	Requirement Name	Requirement Description
1	Fast Loading	The software must load quickly with minimal lag.
2	High Performance	The software should operate smoothly and handle processes efficiently.
3	High Scalability	The software must remain stable even when user traffic increases significantly.
4	Accuracy	The software must operate without calculation or transaction errors.
5	Device Compatibility	The software must run well on various devices including Android, iOS, Windows, Linux.

No.	Requirement Name	Requirement Description
6	User-Friendly Interface	The software should have an intuitive and easy-to-use UI.
7	Easy Installation	The software must be easy to install with minimal setup steps.

3.2 System design

3.2.1 Database design

a) Account

Table 3.4 Account table

No.	Field Name	Data Type	Constraint	Description
1	AccountID	int	Primary key	Account ID
2	Username	Varchar(255)		Login name
3	Password	Varchar(255)		Password
4	AuthenticatedEmail	Varchar(255)		Registered email
5	Role	Varchar(255)		User role
6	DateOfRegistry	date		Registration date
7	RememberLogin	bit		Remember login status
8	Online	bit		Online status

b) Product*Table 3.5 Product table*

No.	Field Name	Data Type	Constraint	Description
1	ProductID	int	Primary key	Product ID
2	ProductName	Varchar(255)	Not null	Product name
3	Description	Varchar(255)	Not null	Product description
4	Price	string	Not null	Product price
5	CreateDate	date	Not null	Creation date
6	Quantity	int	Not null	Product quantity
7	AttributeList	Varchar(255)		Product attributes
8	ExtraImageList	Varchar(255)		Extra images
9	MainImage	Varchar(255)		Main image
10	BannedState	bit		Banned status
11	CategoryID	int	Foreign key, Not null	Category ID
12	ShopID	int	Foreign key, Not null	Shop ID
13	ReviewState	bit	Foreign key	Review state

c) Shop

Table 3.6 Shop table

No.	Field Name	Data Type	Constraint	Description
1	ShopID	int	Primary key	Shop ID
2	Name	Varchar(255)	Not null	Shop name
3	Description	Varchar(255)	Not null	Description
4	Address	Varchar(255)	Not null	Address
5	PhoneNumber	Varchar(255)	Not null	Phone number
6	Image	Varchar(255)		Shop image
7	Owner	int	Foreign key, Not null	Shop owner

d) Comment

Table 3.7 Comment table

No.	Field Name	Data Type	Constraint	Description
1	CommentID	int	Primary key	Comment ID
2	OrderDetailID	int	Foreign key, Not null	Order detail ID
3	CustomerID	int	Foreign key, Not null	Customer ID
4	Stars	Tinyint	Not null	Rating stars
5	Content	Varchar(255)	Not null	Comment content
6	DateOfComment	Date	Not null	Comment date
7	ResponseComment	int	Foreign key	Parent comment ID

e) Voucher*Table 3.8 Voucher table*

No.	Field Name	Data Type	Constraint	Description
1	VoucherID	Int	Primary key	Voucher ID
2	VoucherName	Varchar(255)	Not null	Voucher name
3	VoucherType	bit	Not null	Voucher type (0/1)
4	FixedAmount	Varchar(255)	Not null	Fixed discount amount
5	MinAmount	Varchar(255)	Not null	Minimum order amount for discount
6	MaxAmount	Varchar(255)	Not null	Maximum discount amount
7	Percentage	Varchar(255)	Not null	Discount percentage
8	Quantity	Int	Not null	Number of vouchers
9	StartedDay	Date	Not null	Start date
10	EndedDay	Date	Not null	End date
11	ShopID	Int	Foreign key, Not null	Shop ID

f) Order*Table 3.9 Oder table*

No.	Field Name	Data Type	Constraint	Description
1	OrderID	Int	Primary key	Order ID
2	CustomerID	Int	Foreign key, Not null	Customer ID
3	TotalAmount	String	Not null	Total amount
4	DateOrder	Date	Not null	Order date
5	CustomerOrderState	Bit	Not null	Customer order state
6	OrderConfirmState	Bit	Not null	Order confirmation state
7	ReceivedState	Bit	Not null	Received state

g) Item*Table 3.10 Item table*

No.	Field Name	Data Type	Constraint	Description
1	OrderDetailID	Int	Primary key	Order detail ID
2	OrderID	Int	Foreign key, Not null	Order ID
3	ProductID	Int	Foreign key, Not null	Product ID
4	Quantity	Int	Not null	Quantity of products
5	UnitPrice	String	Not null	Unit price of product

No.	Field Name	Data Type	Constraint	Description
6	OrderDetailConfirmationState	Bit	Not null	Order detail confirmation state
7	Discount	Int	Not null	Discount amount
8	VoucherID	Int	Foreign key	Voucher applied

h) Category

Table 3.11 Category table

No.	Field Name	Data Type	Constraint	Description
1	CategoryID	Int	Primary key	Category ID
2	CategoryName	Varchar(255)	Not null	Category name
3	AncestorID	Int	Foreign key	Parent category ID
4	AttributeList	Varchar(255)		Attribute list

CHAPTER 4: WEBSITE IMPLEMENTATION

4.1 User features implementation

1. Homepage

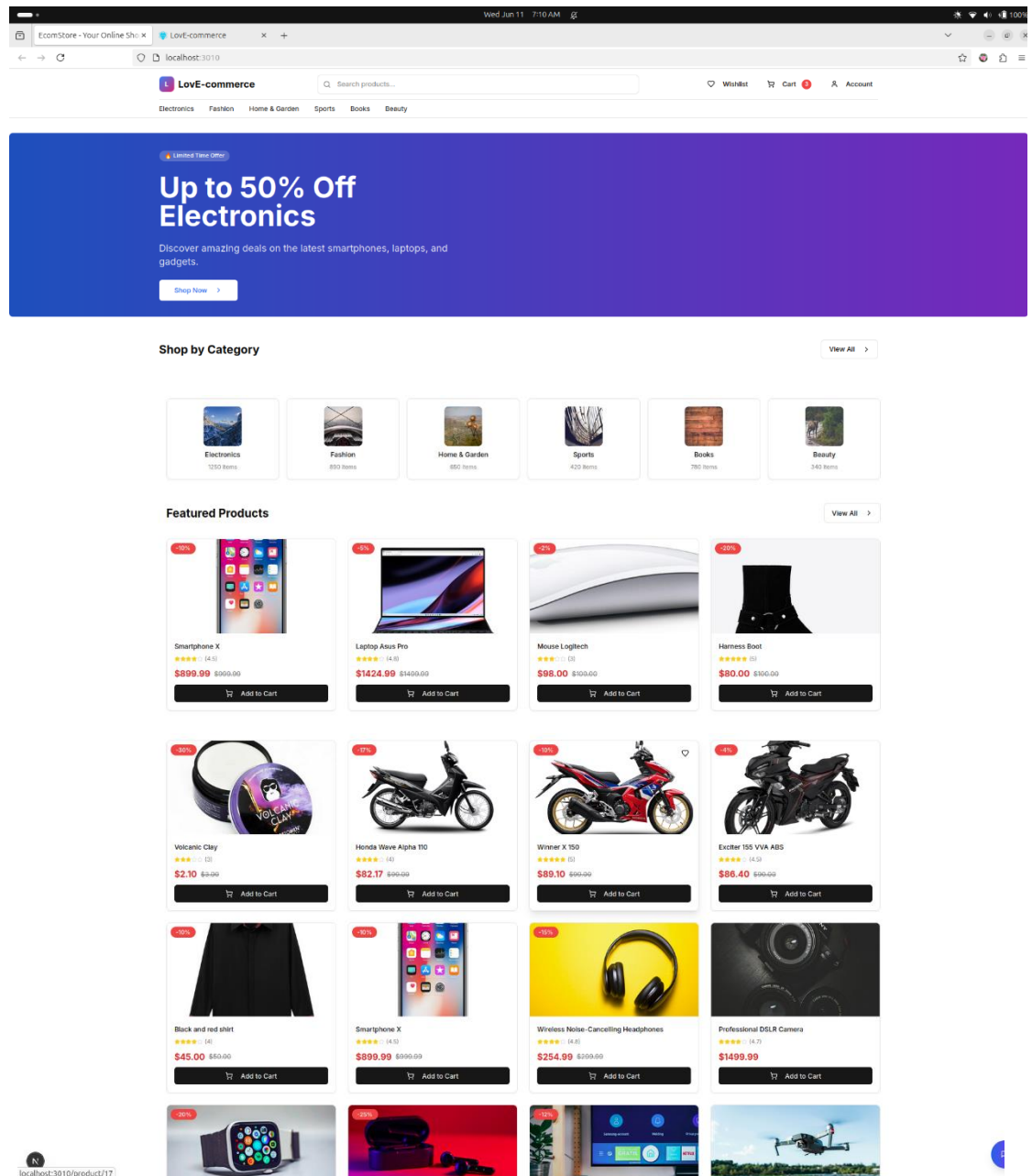


Figure 4.1 Homepage User Page

2. Login

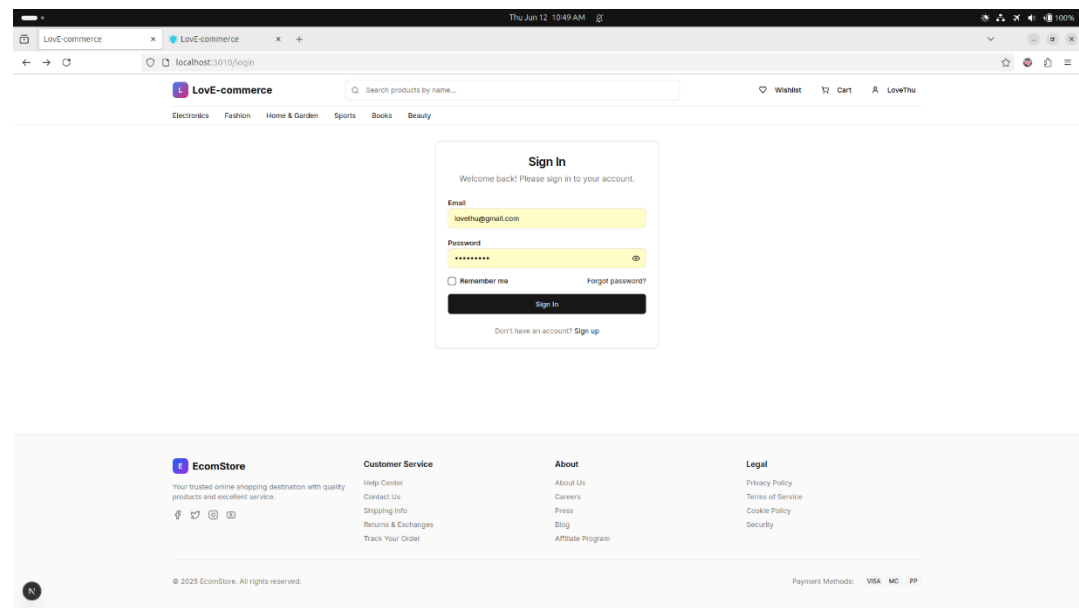


Figure 4.2 Login User Page

3. Register

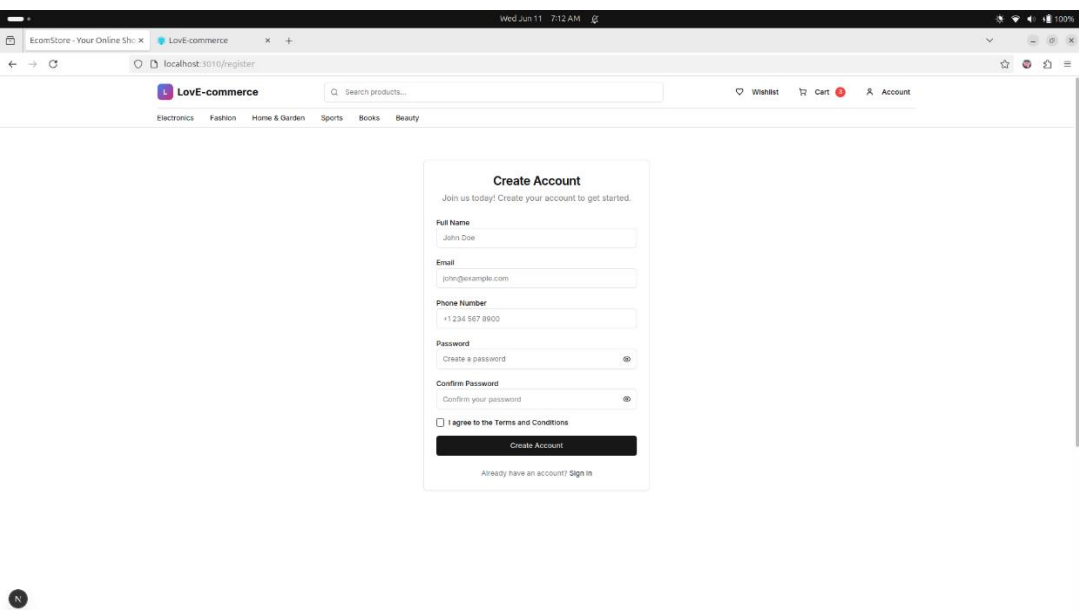


Figure 4.3 Register User Page

4. Cart

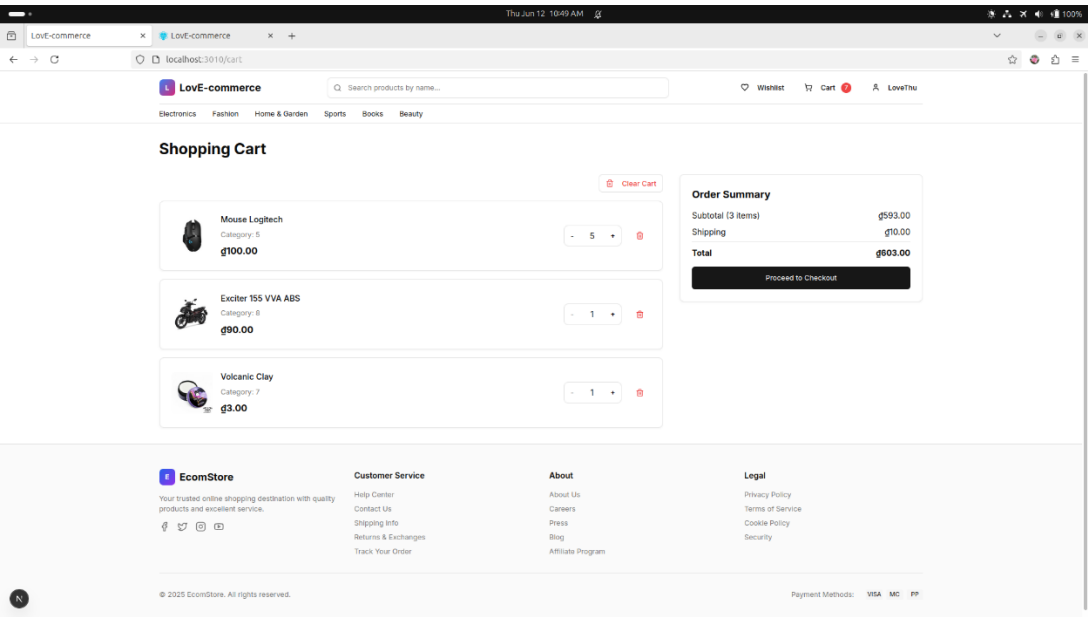


Figure 4.4 Cart User Page

5. Account

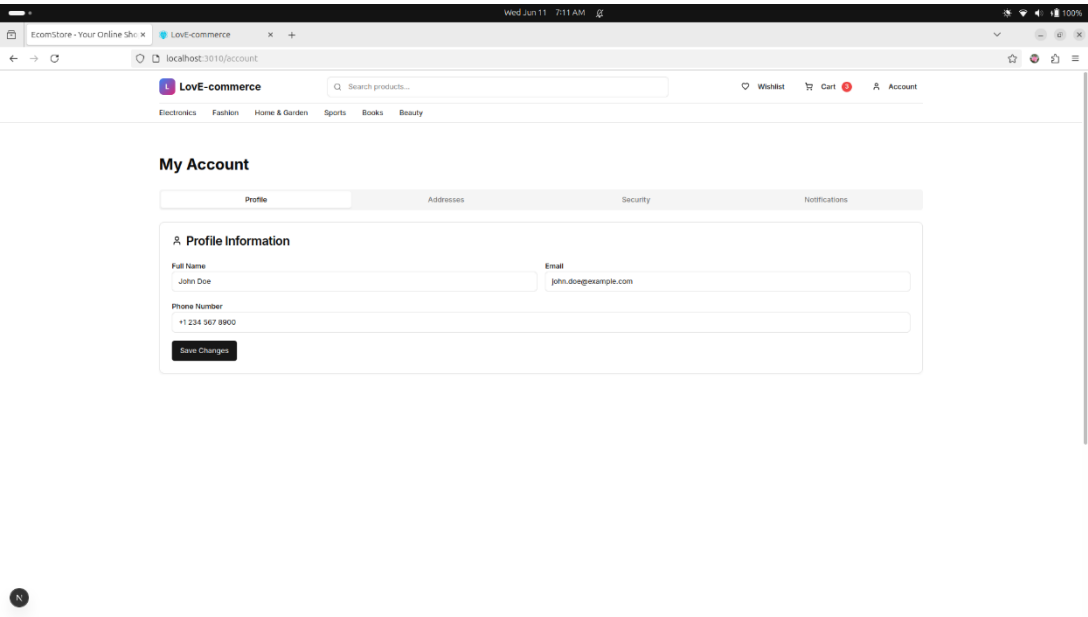


Figure 4.5 Account User Page

6. Search

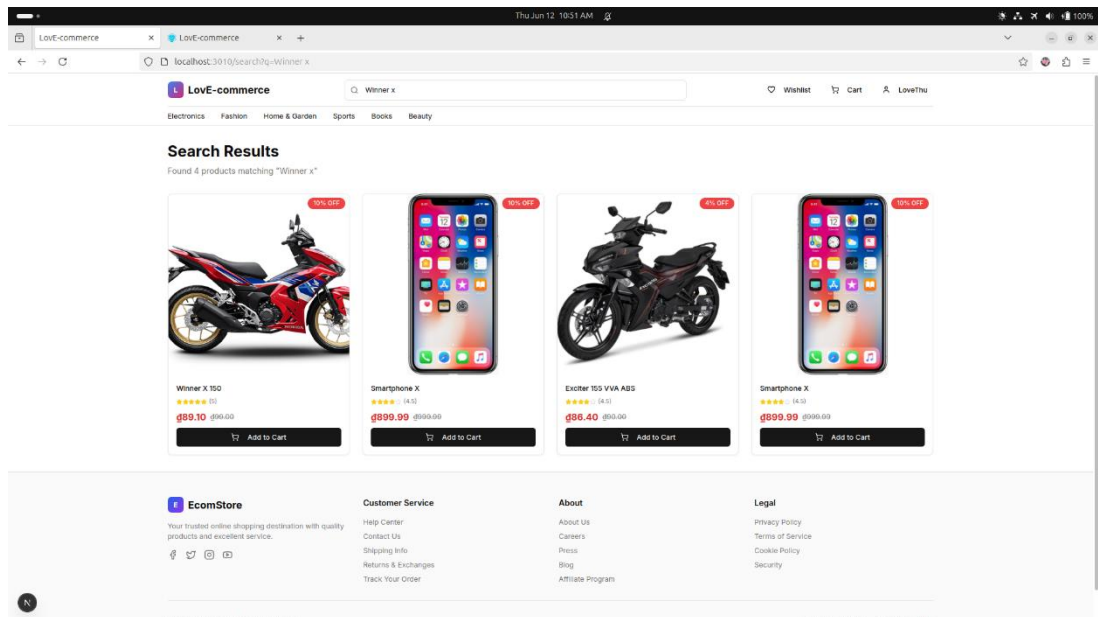


Figure 4.6 Search User Page

7. Checkout

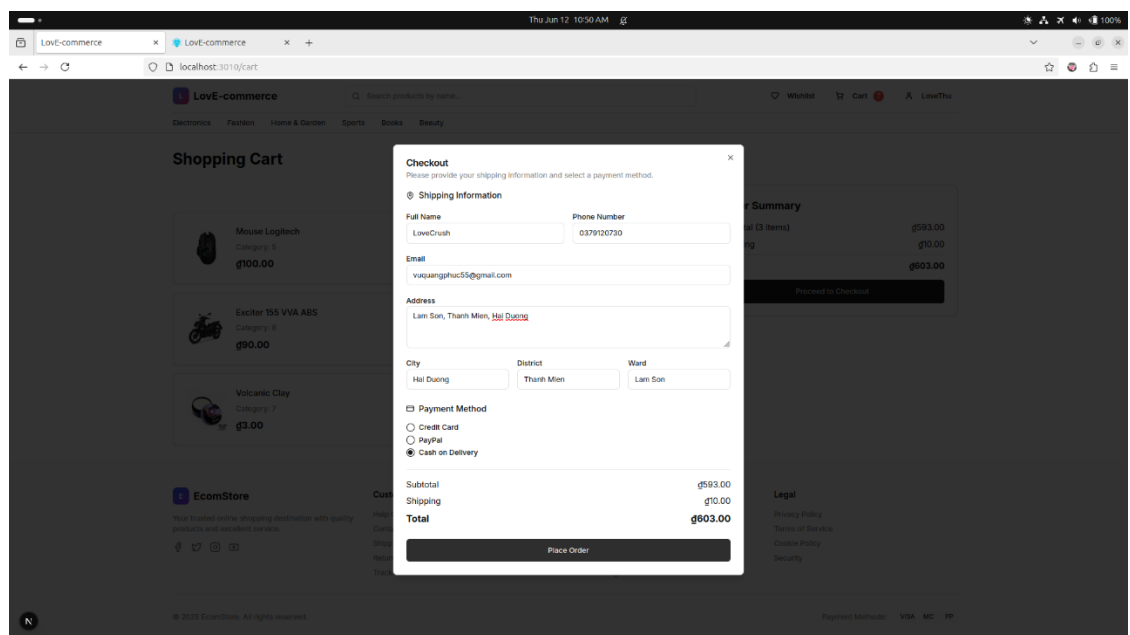


Figure 4.7 Checkout User Page

8. Bill

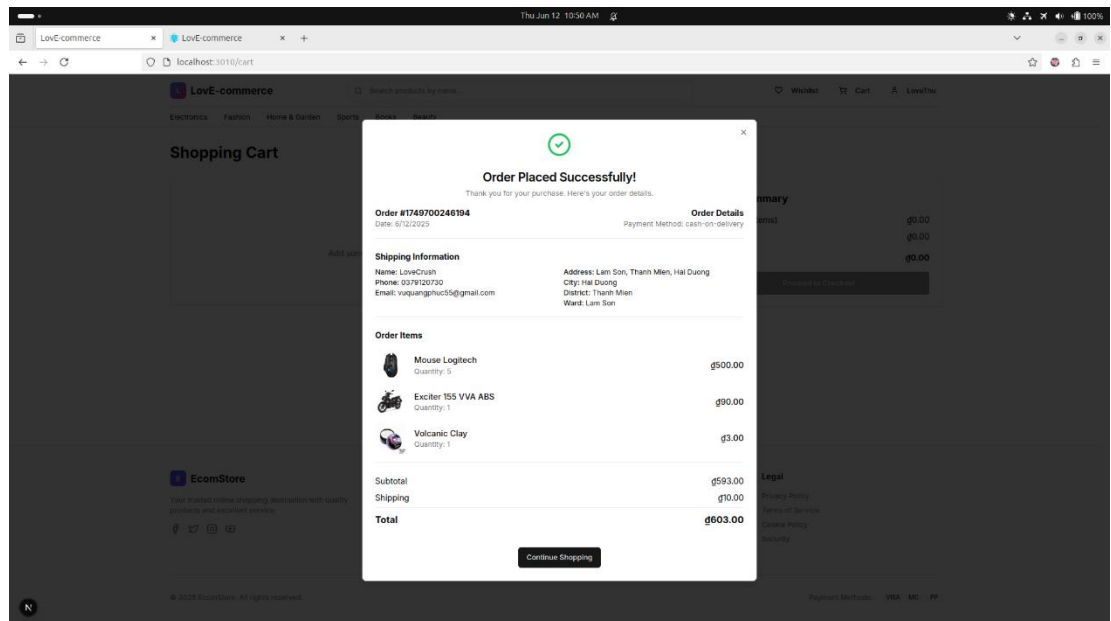


Figure 4.8 Bill User Page

9. Feedback

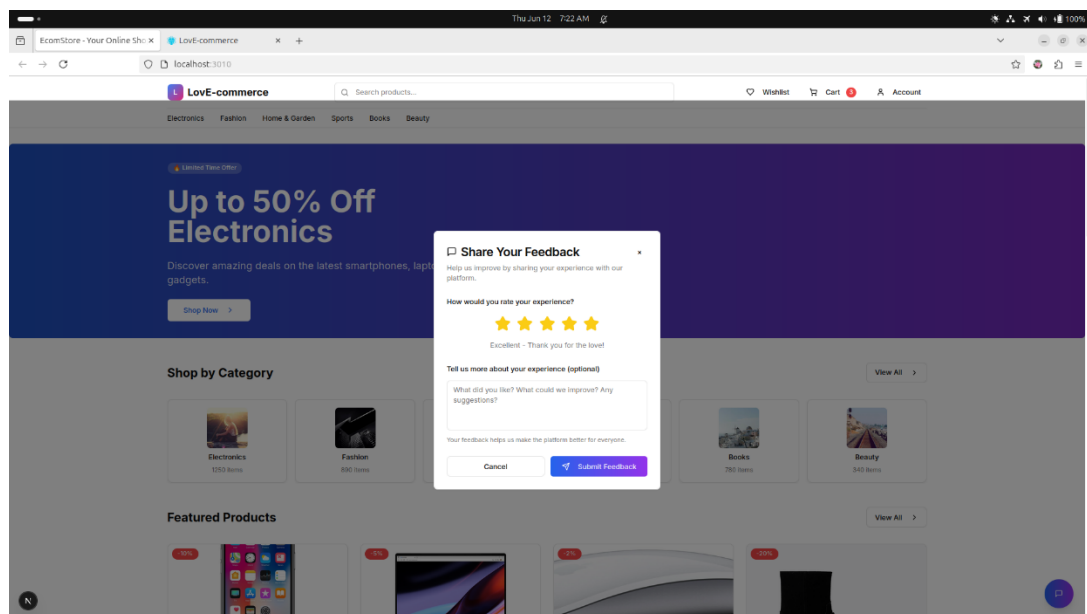


Figure 4.9 Feedback User Page

10. About

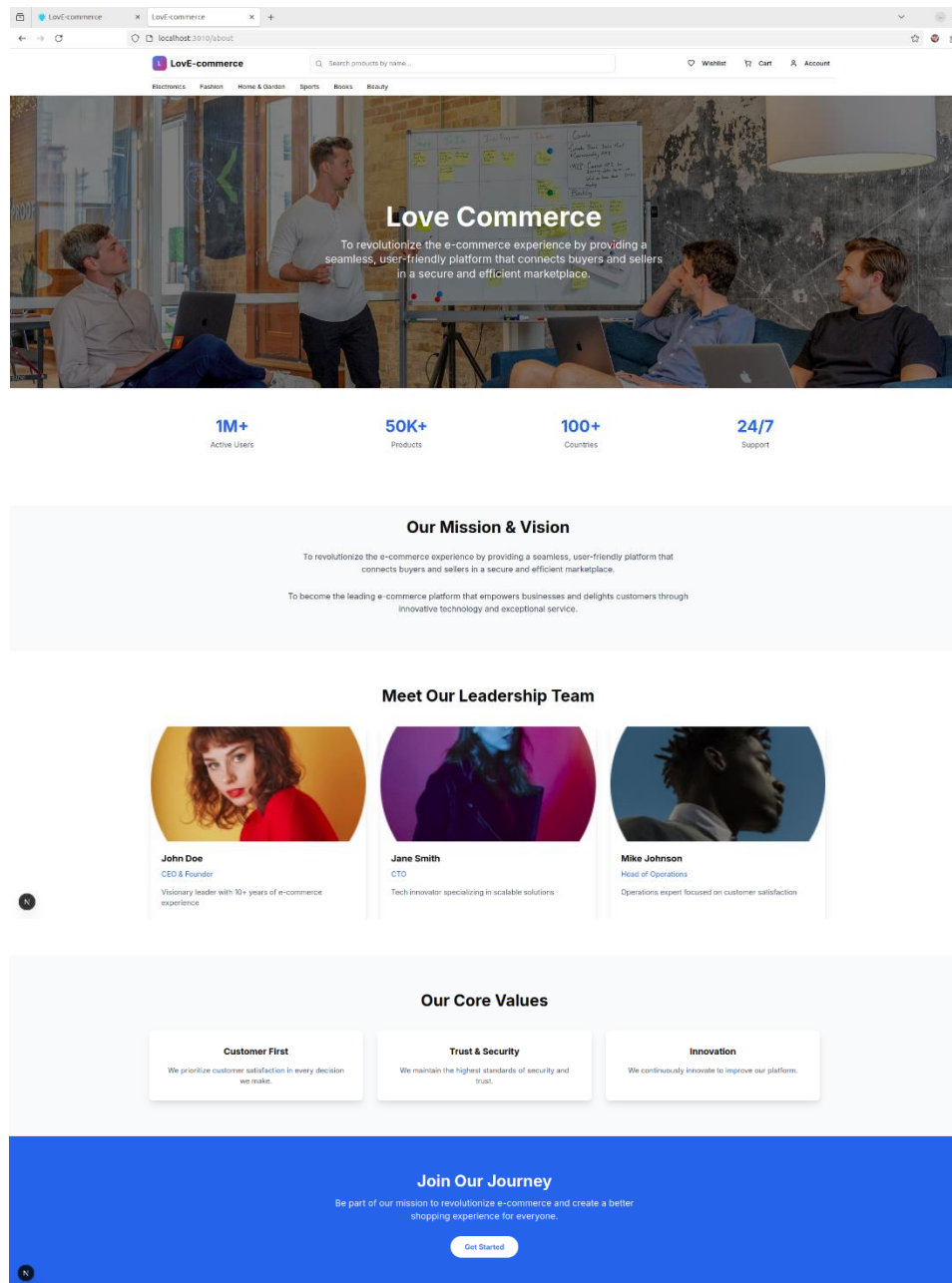


Figure 4.10 About Page

4.2 Admin features implementation

1. Product management

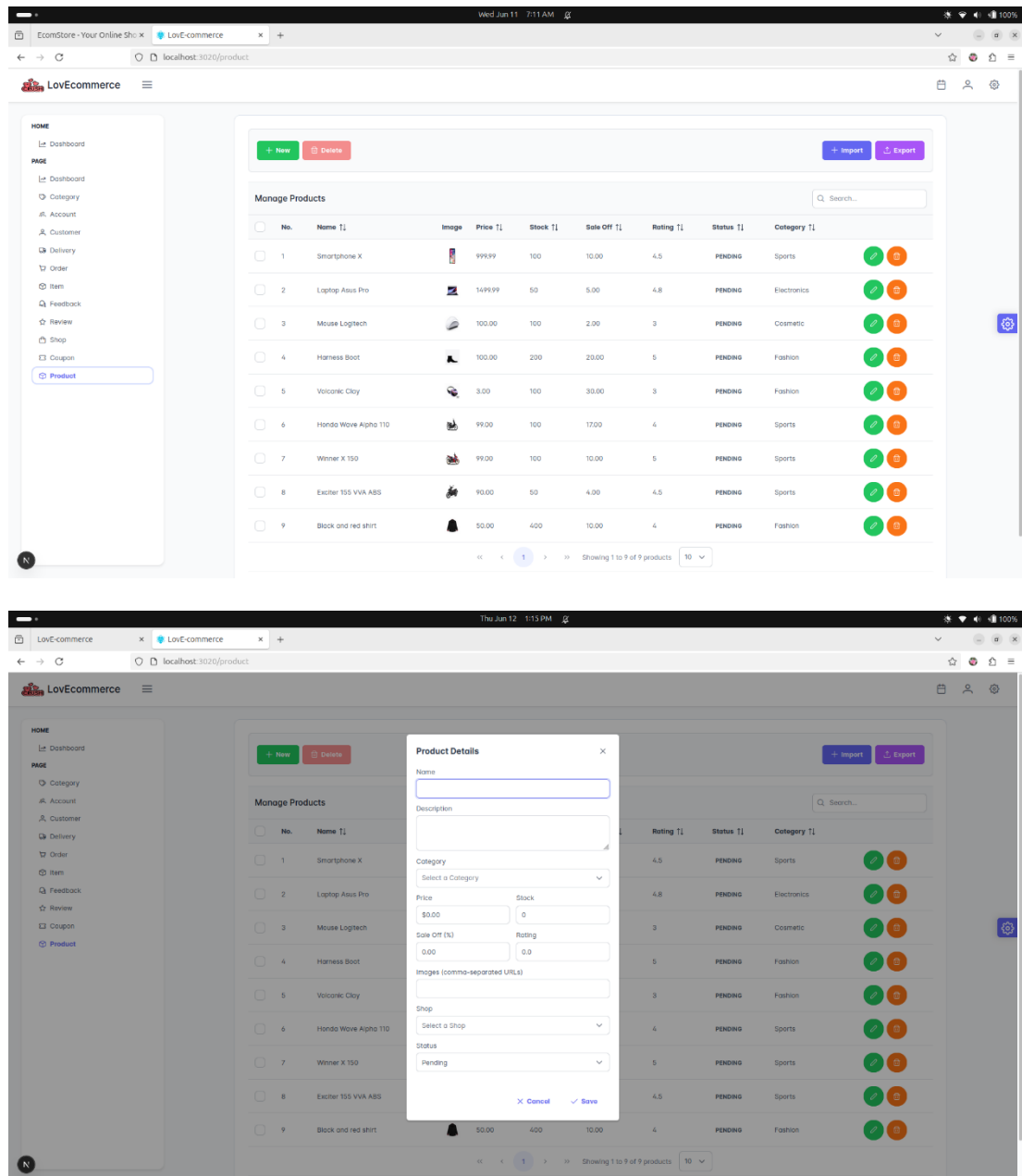


Figure 4.11 Product management GUI

2. Coupon management

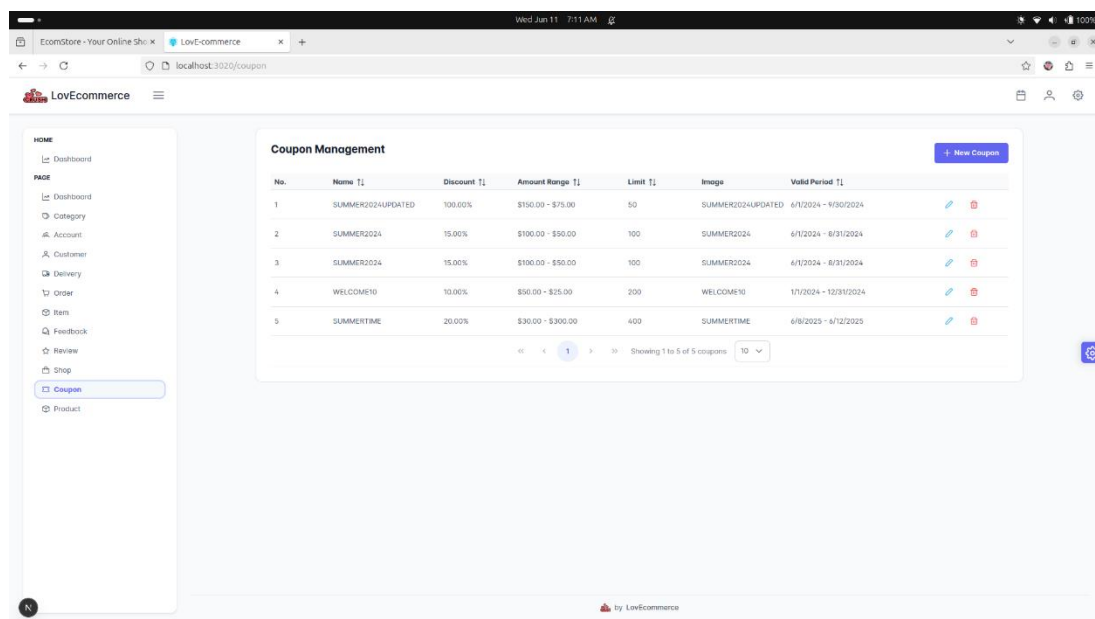


Figure 4.12 Coupon management GUI

3. Feedback management

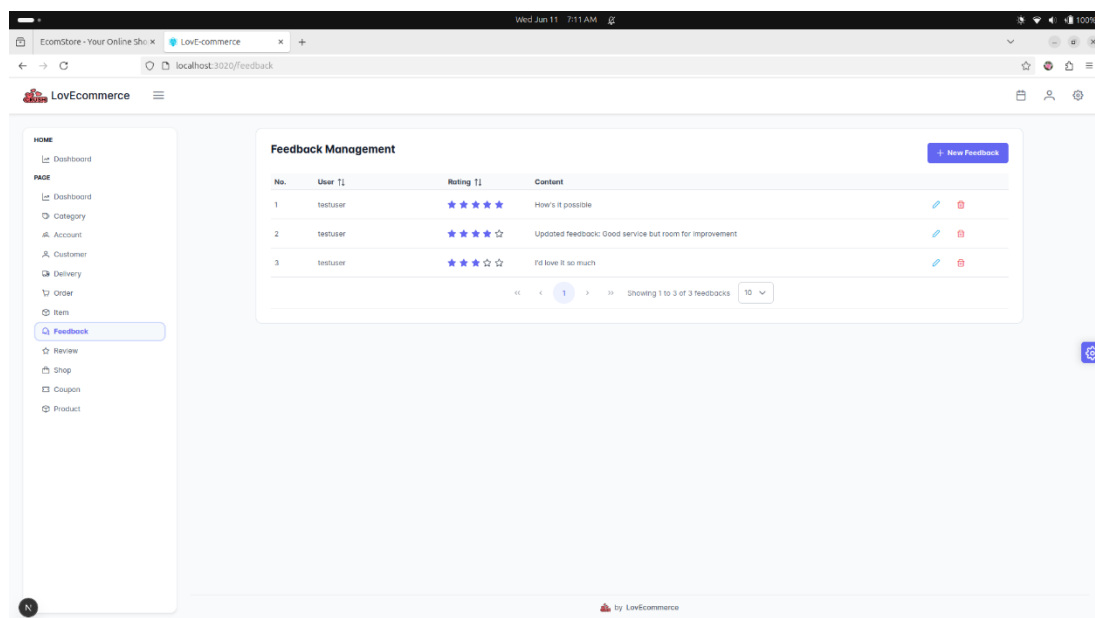


Figure 4.13 Feedback management GUI

4. Delivery Management

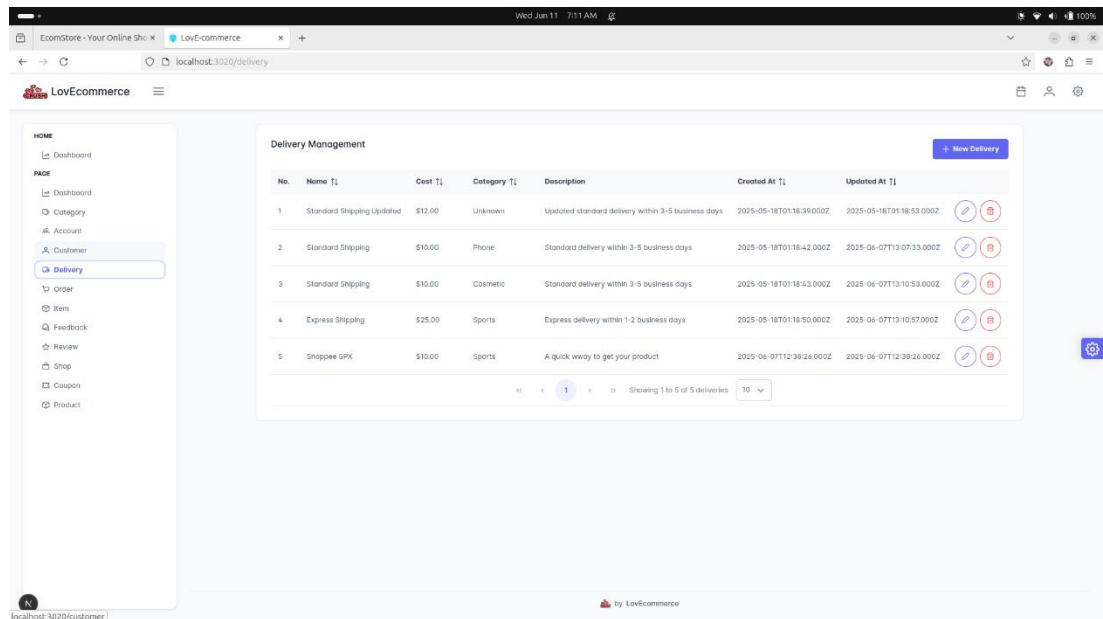


Figure 4.14 Delivery management GUI

5. Account management

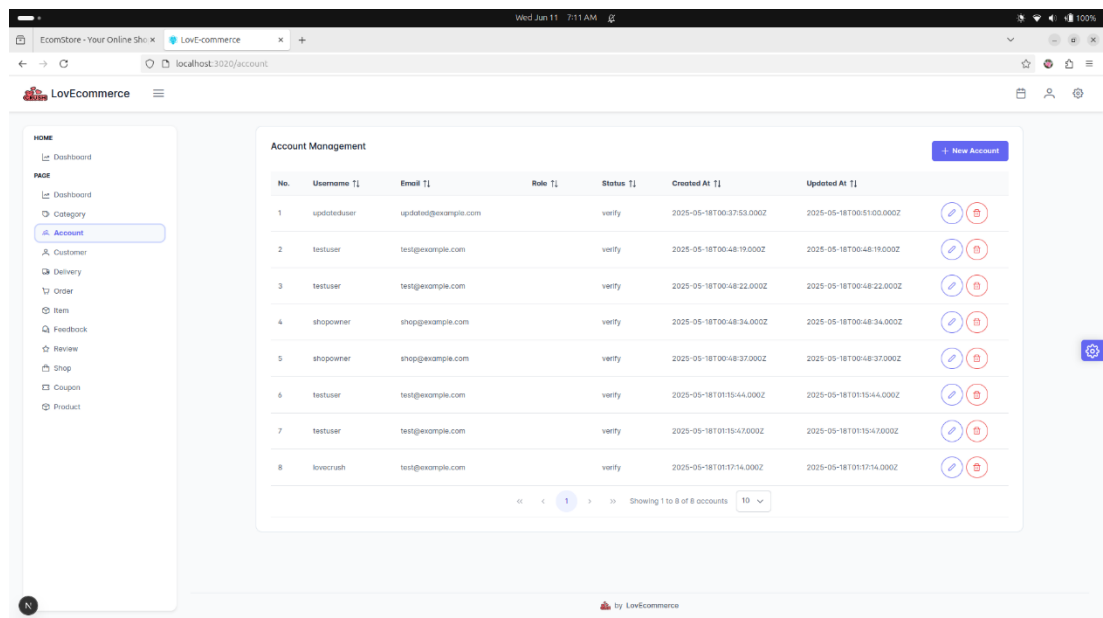


Figure 4.15 Account management GUI

6. Category management

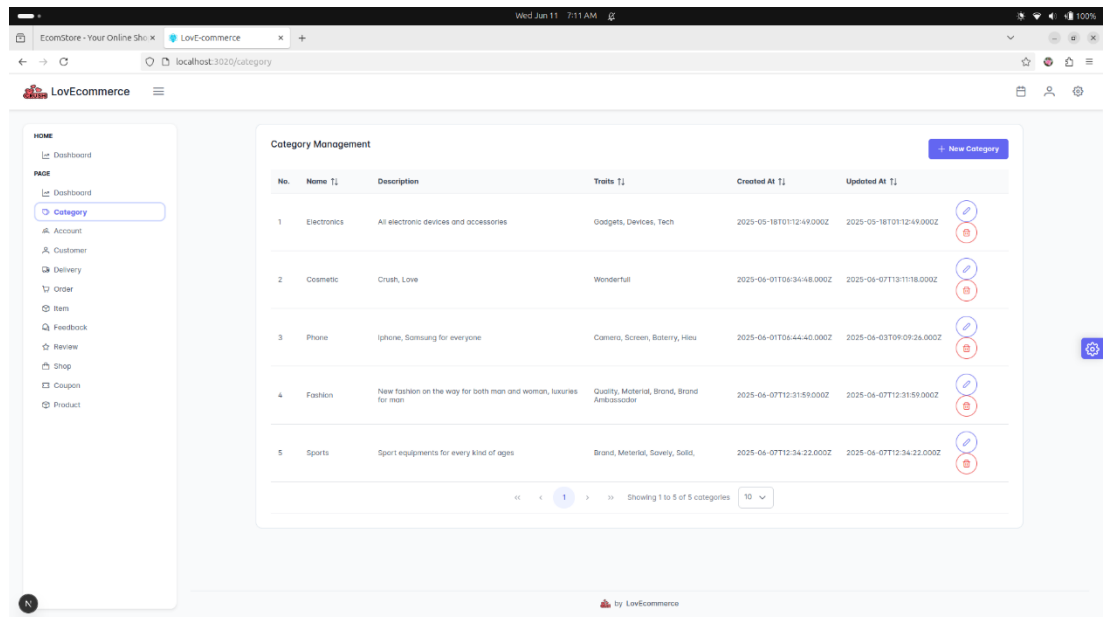


Figure 4.16 Category management GUI

7. Statistic

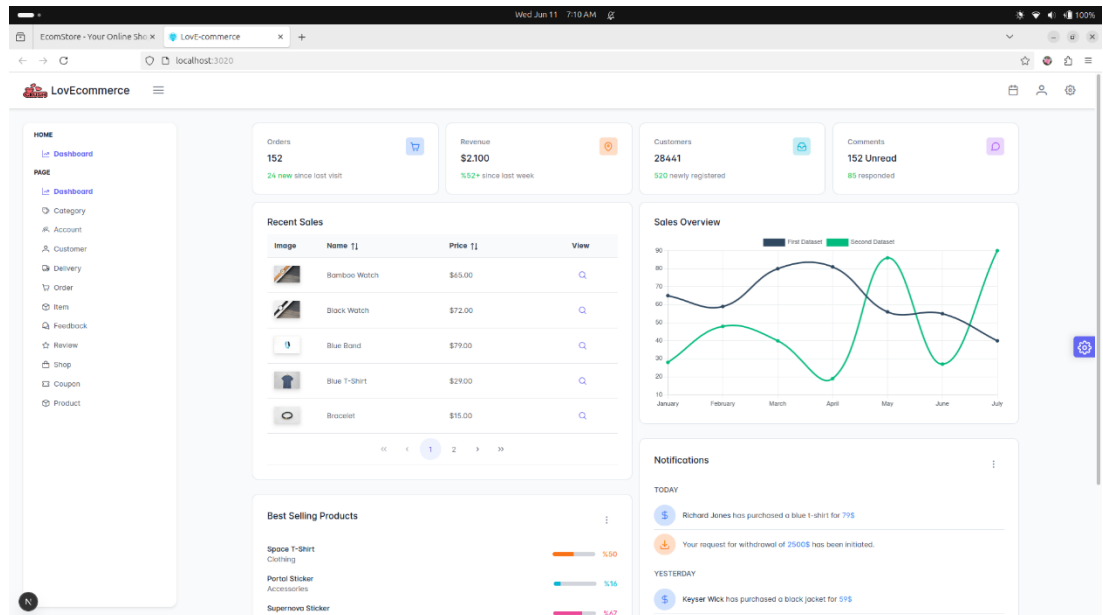


Figure 4.17 Statistic GUI

4.3 Testing and Deploying the Application

4.3.1 Testing

4.3.2 Deployment

To ensure the proper deployment and operation of the e-commerce platform, both hardware and software resources must meet the minimum recommended specifications. These requirements are critical to ensure stable performance, support concurrent user interactions, and facilitate the development and testing process.

➤ *Hardware Requirements*

The following hardware specifications are recommended for hosting and maintaining the backend services, source code, and related static assets of the e-commerce platform:

- A basic server equipped with a mid-range CPU (e.g., Intel i5, AMD Ryzen 5 or higher).
- RAM: Minimum of 8 GB, preferably 16 GB, to ensure smooth handling of background processes, database operations, and concurrent server tasks.
- Storage: Sufficient disk space (SSD preferred) to store:
- Stable internet connection with sufficient upload and download bandwidth to support the projected number of concurrent users.
- Network bandwidth should be scalable depending on user traffic peaks (e.g., during sales or promotional events).

➤ *Software Requirements*

The development, testing, and access to the e-commerce system require the following software components:

- Google Chrome
- Microsoft Edge
- Mozilla Firefox
- (Other Chromium-based browsers are also acceptable)

CONCLUSION

Achievements:

- Developed a complete and functional e-commerce platform management system covering product, order, user, and admin management.
- Successfully applied web development skills including front-end, back-end, database design, and UML modeling.
- Improved operational efficiency by optimizing inventory, supplier coordination, and customer service.
- Followed a structured software development life cycle ensuring quality and functionality.

Limitations:

- Limited development time (3 months) restricted full feature completion and refinement.
- Some design, coding, and testing aspects may lack depth due to limited experience.
- Performance optimization and security measures need further enhancement.
- Advanced features like payment integration and logistics management are not fully implemented.

Improvements

- Extend development and testing time to add advanced features and improve system robustness.
- Enhance security and performance through rigorous testing and optimization.
- Improve user interface and experience based on real user feedback.
- Adopt modern frameworks and automation tools for better development efficiency.
- Explore AI and machine learning to personalize user experience and improve data analysis.

REFERENCES

- [1] Documents and slides of mentor Vu Xuan Thang.